



University
of Exeter

COMM113

Deep Learning – Week 12

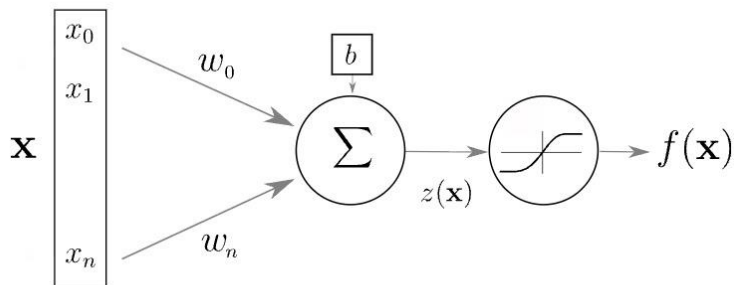
Revision

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Revision

- Neural Networks
- Activation function
- Softmax
- Cross-Entropy
- Backprop
- Regularisers
- Example questions

Neuron



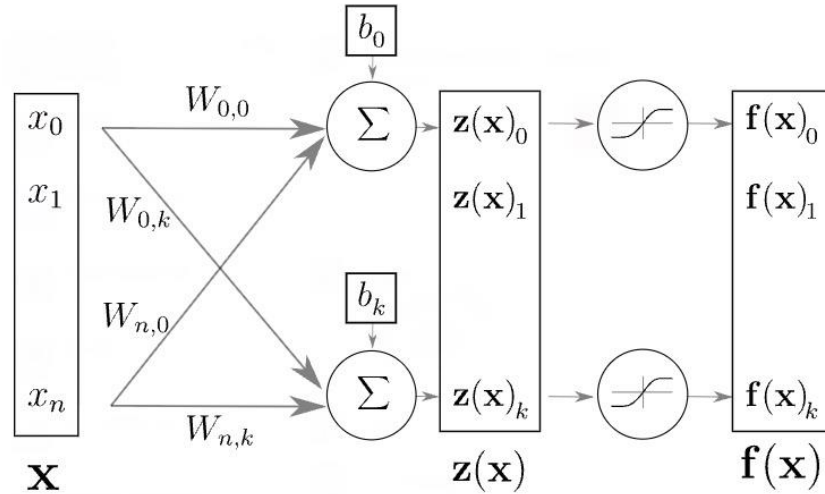
$$z(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

$$f(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x} + b)$$

- $\mathbf{x}, f(\mathbf{x})$ input and output
- $z(\mathbf{x})$ pre-activation
- $\mathbf{w}, b$ weights and bias
- g activation function

This is the fundamental unit: the neuron. It takes weighted inputs, applies a bias, and passes the result through an activation function.

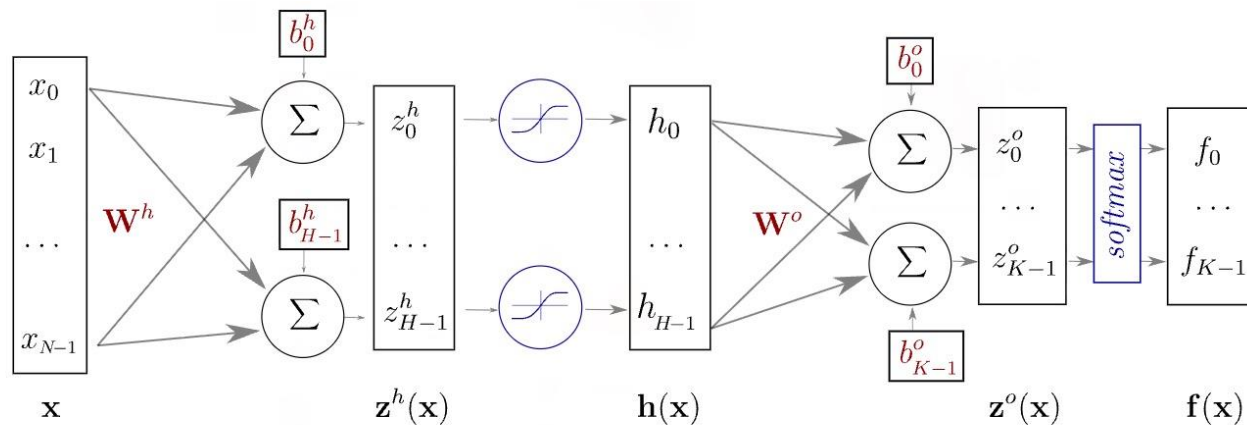
Layer of Neurons



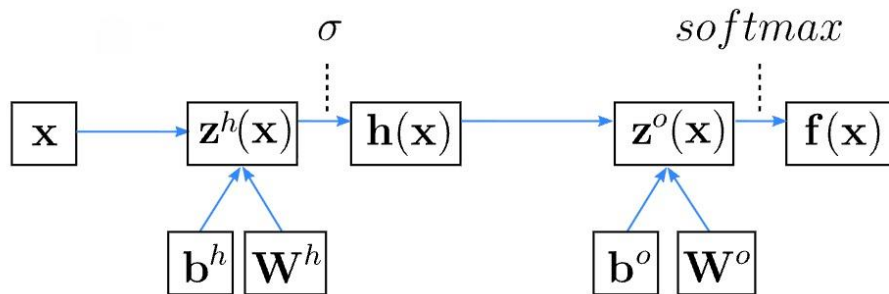
$$\mathbf{f}(\mathbf{x}) = g(\mathbf{z}(\mathbf{x})) = g(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- $\mathbf{W}, \mathbf{b}$ now matrix and vector

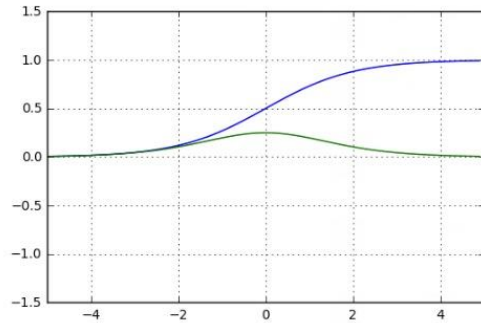
One Hidden Layer Network



Alternate representation

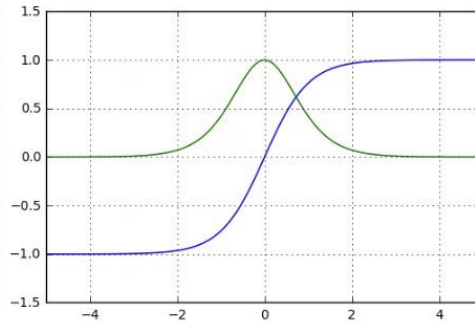


Activation Functions



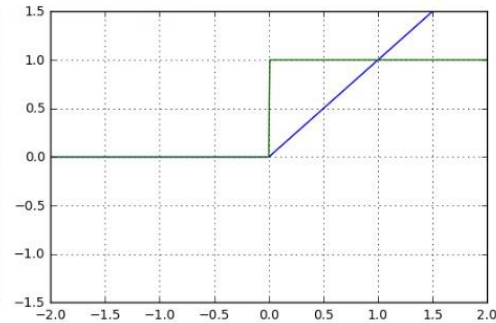
$$\text{sigm}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{sigm}'(x) = \text{sigm}(x)(1 - \text{sigm}(x))$$



$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

$$\tanh'(x) = 1 - \tanh(x)^2$$



$$\text{relu}(x) = \max(0, x)$$

$$\text{relu}'(x) = 1_{x>0}$$

- blue: activation function
- green: derivative

Enhance expression ability

Softmax

$$\text{softmax}(\mathbf{x}) = \frac{1}{\sum_{i=1}^n e^{x_i}} \cdot \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ \vdots \\ e^{x_n} \end{bmatrix}$$

$$\frac{\partial \text{softmax}(\mathbf{x})_i}{\partial x_j} = \begin{cases} \text{softmax}(\mathbf{x})_i \cdot (1 - \text{softmax}(\mathbf{x})_i) & i = j \\ -\text{softmax}(\mathbf{x})_i \cdot \text{softmax}(\mathbf{x})_j & i \neq j \end{cases}$$

A softmax layer in a neural network takes raw scores (logits) from the previous layer and **transforms them into a probability distribution**, where each output value is between 0 and 1 and the sum of all values equals 1.

Cross-entropy Loss

$$L = - \sum_{i=1}^N y_i \log(p_i)$$

$$\frac{\partial L}{\partial p_i} = -\frac{y_i}{p_i}, \quad i = 1, \dots, N.$$

Interpretation

- If class i is the true label ($y_i = 1$):

$$\frac{\partial L}{\partial p_i} = -\frac{1}{p_i}.$$

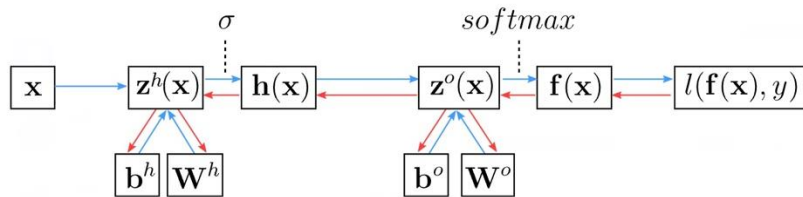
- If class i is not the true label ($y_i = 0$):

$$\frac{\partial L}{\partial p_i} = 0.$$

Backprop gradients

Compute activation gradients

- $\nabla_{\mathbf{z}^o(\mathbf{x})} l = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$



Compute layer params gradients

- $\nabla_{\mathbf{W}^o} l = \nabla_{\mathbf{z}^o(\mathbf{x})} l \cdot \mathbf{h}(\mathbf{x})^\top$
- $\nabla_{\mathbf{b}^o} l = \nabla_{\mathbf{z}^o(\mathbf{x})} l$

Compute prev layer activation gradients

- $\nabla_{\mathbf{h}(\mathbf{x})} l = \mathbf{W}^{o\top} \nabla_{\mathbf{z}^o(\mathbf{x})} l$
- $\nabla_{\mathbf{z}^h(\mathbf{x})} l = \nabla_{\mathbf{h}(\mathbf{x})} l \odot \sigma'(\mathbf{z}^h(\mathbf{x}))$

Practice

- $x \in \mathbb{R}^2$, one hidden layer ($m = 2$), $N = 3$ classes
- Activation $\sigma = \text{ReLU}$; output uses **softmax**
- Cross-entropy loss $\ell(f(x), y) = -\sum_{i=1}^N y_i \log f_i(x)$

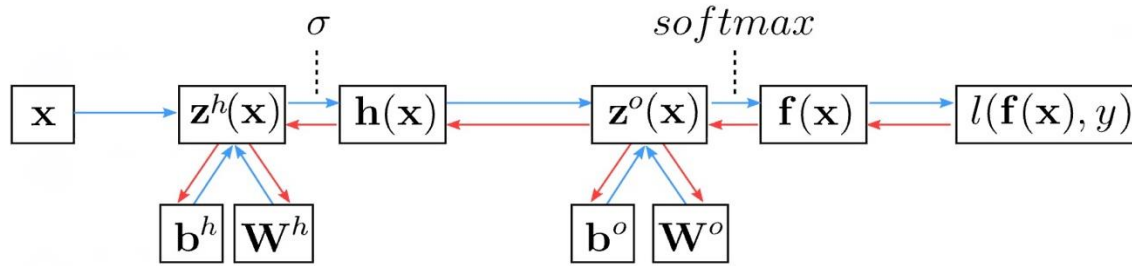
Parameters

$$W^h = \begin{bmatrix} 0.2 & -0.4 \\ 0.7 & 0.5 \end{bmatrix}, \quad b^h = \begin{bmatrix} 0.1 \\ -0.3 \end{bmatrix}$$

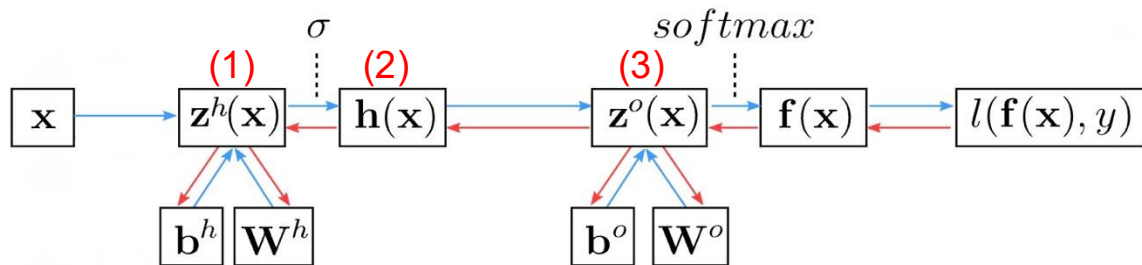
$$W^o = \begin{bmatrix} 0.6 & -0.1 \\ -0.2 & 0.9 \\ 0.1 & 0.2 \end{bmatrix}, \quad b^o = \begin{bmatrix} 0.05 \\ -0.05 \\ 0.02 \end{bmatrix}$$

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad (\text{true class 1})$$

Practice - Forward Pass



Practice - Forward Pass

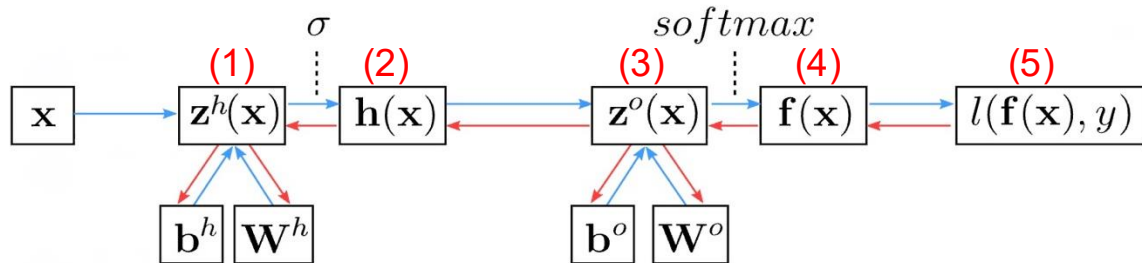


$$z^h = W^h x + b^h = \begin{bmatrix} -0.5 \\ 1.4 \end{bmatrix} \quad (1)$$

$$h = \sigma(z^h) = \max(0, z^h) = \begin{bmatrix} 0 \\ 1.4 \end{bmatrix} \quad (2)$$

$$z^o = W^o h + b^o = \begin{bmatrix} -0.09 \\ 1.21 \\ 0.30 \end{bmatrix} \quad (3)$$

Practice - Forward Pass



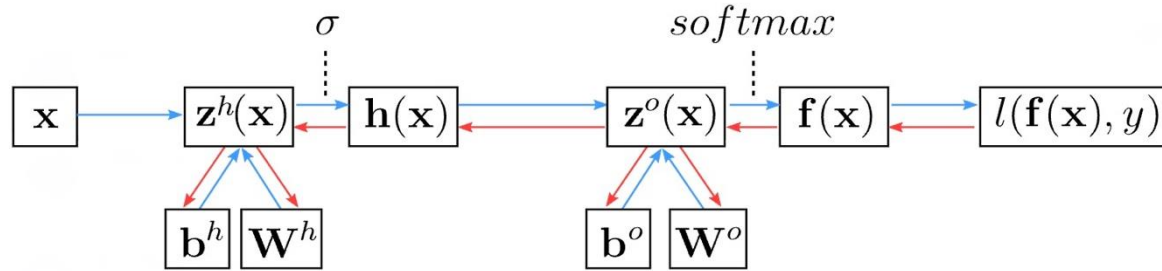
Softmax:

$$\mathbf{f} = \text{softmax}(\mathbf{z}^o) = \begin{bmatrix} 0.1627 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (4)$$

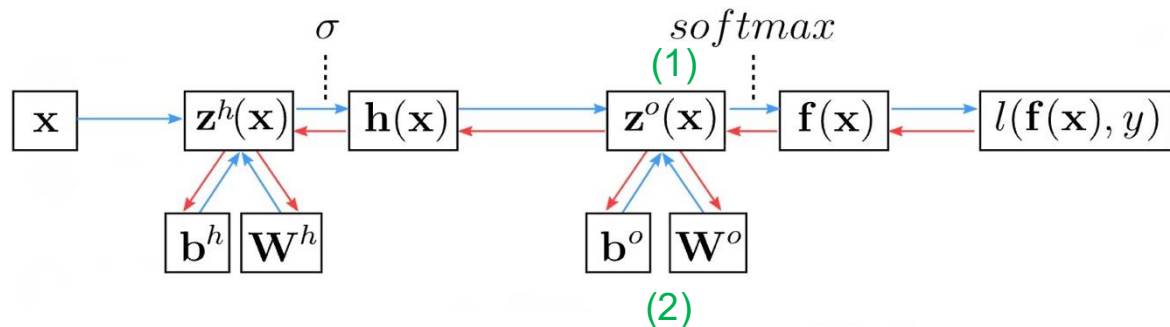
Loss:

$$\ell = -\log f_1 \approx 1.8158 \quad (5)$$

Practice – Backward Pass



Practice – Backward Pass



Output

$$\delta^o = \frac{\partial \ell}{\partial z^o} = f - y = \begin{bmatrix} -0.8373 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (1)$$

$$\nabla_{z^o(\mathbf{x})} l(\mathbf{f}(\mathbf{x}), y) = \mathbf{f}(\mathbf{x}) - \mathbf{e}(y)$$

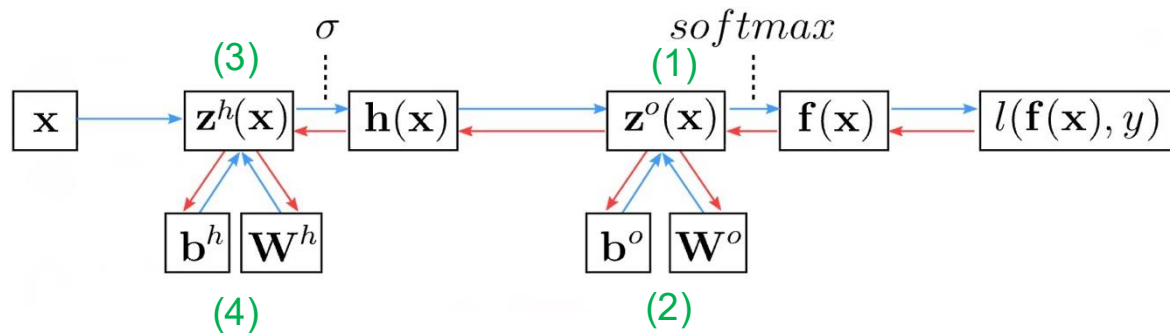
$\mathbf{e}(y)$: one-hot encoding of y

Output-layer gradients

$$\nabla_{W^o} \ell = \delta^o \mathbf{h}^\top = \begin{bmatrix} 0 & -1.1722 \\ 0 & 0.8358 \\ 0 & 0.3364 \end{bmatrix}, \quad \nabla_{b^o} \ell = \delta^o = \begin{bmatrix} -0.8373 \\ 0.5970 \\ 0.2403 \end{bmatrix} \quad (2)$$

- $\nabla_{W^o} \ell = \nabla_{z^o(\mathbf{x})} \ell \cdot \mathbf{h}(\mathbf{x})^\top$
- $\nabla_{b^o} \ell = \nabla_{z^o(\mathbf{x})} \ell$

Practice – Backward Pass



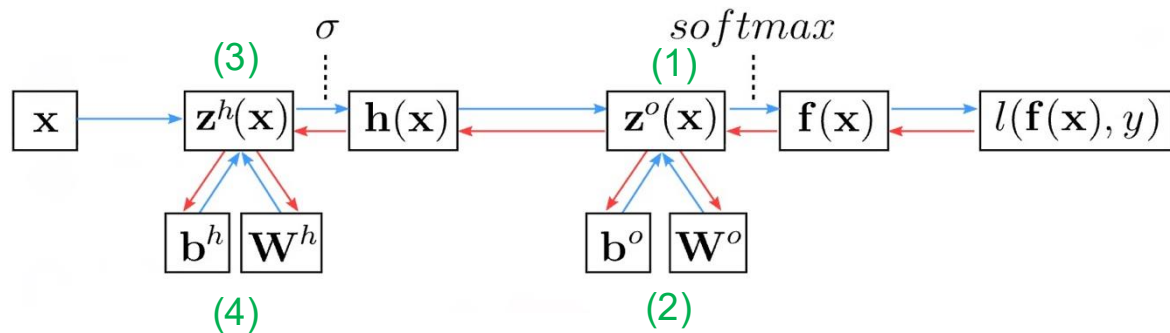
Backprop to hidden

$$\delta^h = \left((W^o)^\top \delta^o \right) \odot \sigma'(z^h) = \begin{bmatrix} 0 \\ 0.6691 \end{bmatrix} \quad (\text{ReLU mask } [0, 1]) \quad (3)$$

Hidden-layer gradients

$$\nabla_{W^h} \ell = \delta^h x^\top = \begin{bmatrix} 0 & 0 \\ 0.6691 & 1.3382 \end{bmatrix}, \quad \nabla_{b^h} \ell = \delta^h = \begin{bmatrix} 0 \\ 0.6691 \end{bmatrix} \quad (4)$$

Practice – Summary



- $\delta^o = f - y$ (1)

- $\nabla_{W^o} = \delta^o h^\top, \nabla_{b^o} = \delta^o$ (2)

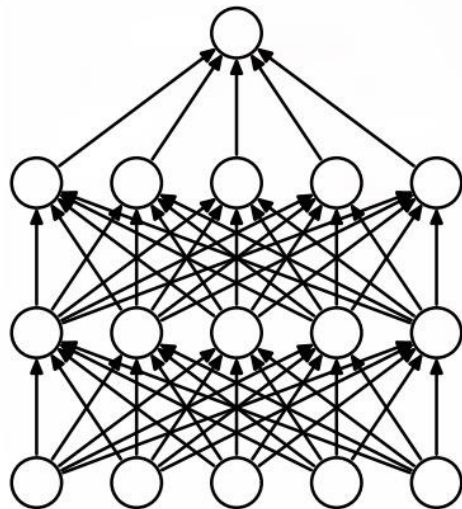
- $\delta^h = (W^o)^\top \delta^o \odot \sigma'(z^h)$ (3)

- $\nabla_{W^h} = \delta^h x^\top, \nabla_{b^h} = \delta^h$ (4)

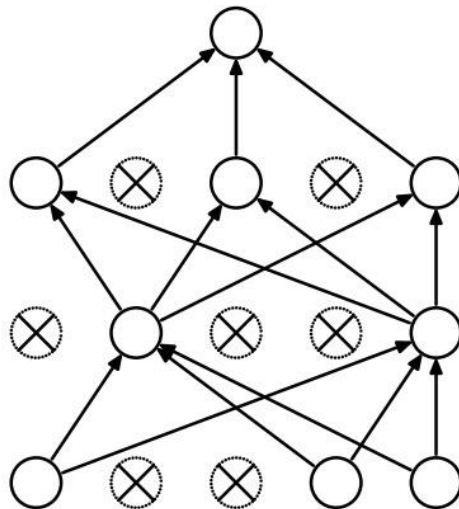
L1 and L2

Property	L1	L2
Penalises	Absolute value	Squared value
Effect	Some weights = 0	All weights shrink
Result	Sparse model	Smooth model
Use case	Feature selection	Prevent overfitting
Geometry	Diamond	Circle

Dropout



(a) Standard Neural Net



(b) After applying dropout.

At each training iteration,
With probability p to
“dropout”

p : dropout rate

Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

A first “deep” regularization technique is dropout (Srivastava et al., 2014). It consists of removing units at random during the forward pass on each sample, and putting them all back during test.

Data Augmentation

Expand the size of a training set by creating modified data from the existing one.



Data Augmentation



Original Image



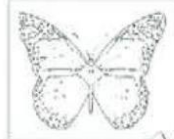
De-texturized



De-colored



Edge Enhanced



Salient Edge Map



Flip/Rotate

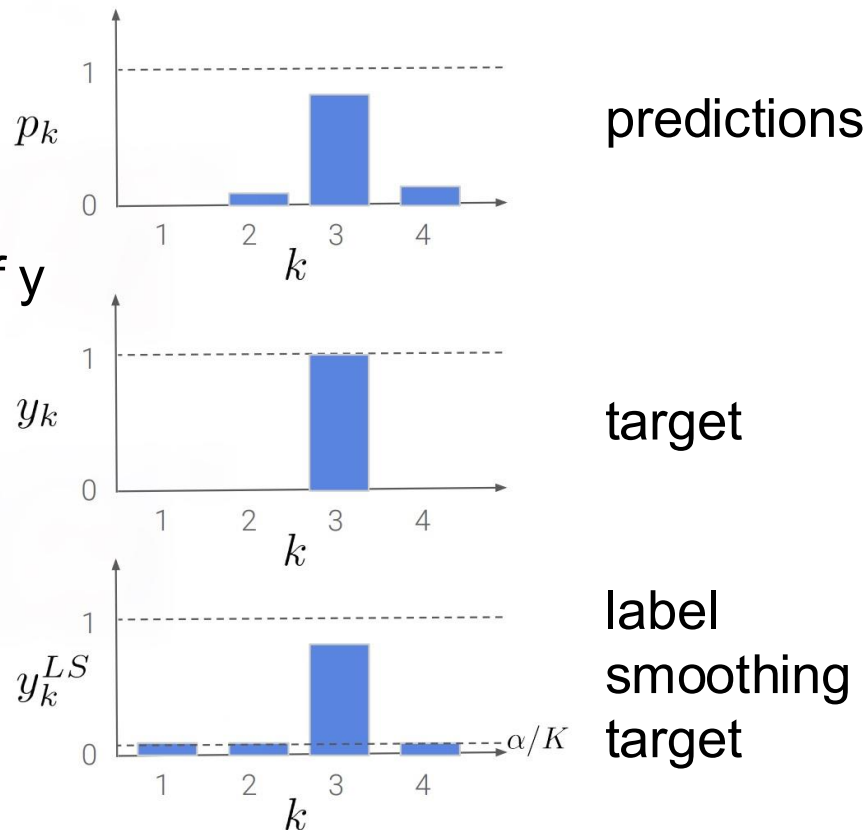
Imagine teaching a model to recognise a cat — showing the same cat from different angles helps it learn.

Label smoothing

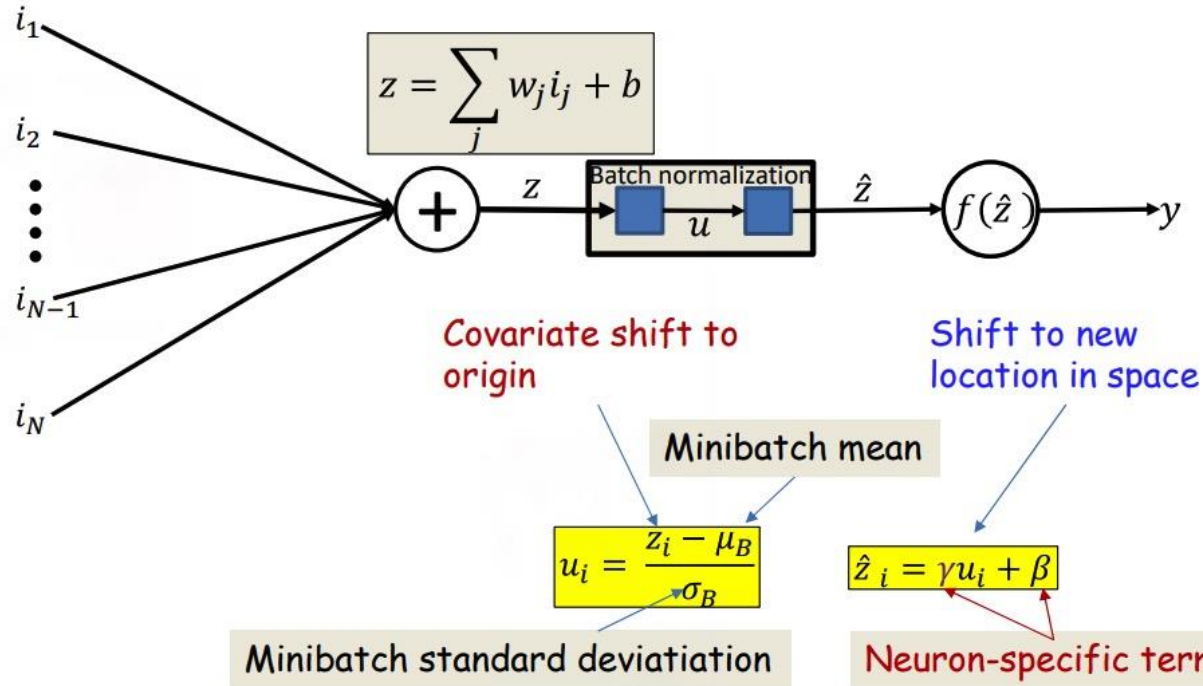
Label smoothing replaces one-hot encoded label vector y with a mixture of y and the uniform distribution

$$y_k^{LS} = y_k(1 - \alpha) + \alpha/K$$

May help generalisation! It depends ...

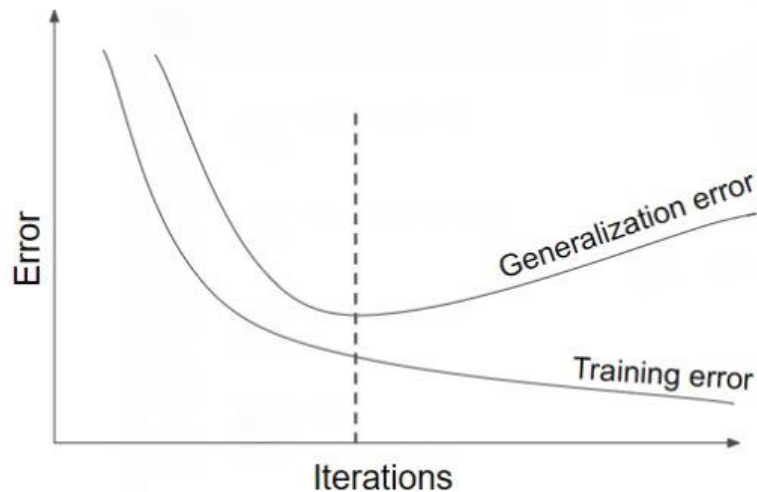


Batch Normalisation



- BN aggregates the statistics over a minibatch and normalises the batch by them
- Normalised instances are “shifted” to a unit-specific location

Early Stopping



Strengths

- Effective and simple
- Applicable to almost any model without adjustment
- Combinable with other regularization methods

Weaknesses

- Periodical evaluation of validation error
- We have to save the whole model each time validation error improves
- Less data for training

Regularisers

Technique	Purpose	When to Use	Typical Placement / Scenario	Example Hyperparams / Notes
L1 Regularisation	Enforce sparsity, feature selection	Sparse models, small datasets	Linear or dense layers	$\lambda = 1\text{e-}5 - 1\text{e-}3$
L2 (Weight Decay)	Control weight magnitude, smooth boundary	Almost always used	All param layers (CNNs, Transformers)	$\lambda = 1\text{e-}5 - 1\text{e-}2$
Dropout	Prevent neuron co-adaptation	Deep/large networks	Between dense layers, before classifier head	$p = 0.3 - 0.5$
Data Augmentation	Increase data diversity	Small or imbalanced datasets	Input preprocessing stage	Random flip, crop, jitter, noise
Label Smoothing	Prevent overconfidence, improve calibration	Noisy / large-class datasets	Classification head with Cross-Entropy	$\epsilon = 0.05 - 0.2$
Early Stopping	Stop before overfitting	Any model with validation set	Training loop monitoring val loss	patience = 3–10 epochs
Batch Normalisation	Stabilise activations, speed up training	Deep CNNs / MLPs	Between linear/conv and ReLU	momentum = 0.1, $\epsilon = 1\text{e-}5$

Example questions

Question 1 — Foundations of Deep Learning (10 marks)

(a) Define deep learning. How does deep learning extend the idea of traditional machine learning?

(5 marks)

(b) Give one example where deep learning fails compared with traditional methods, and explain why.

(5 marks)

Example questions

Question 3 — Regularisation in Deep Learning (20 marks)

Discuss the following regularisation techniques and explain when each is most appropriate:

(a) Data augmentation

(5 marks)

(b) Early stopping

(5 marks)

(c) Label smoothing

(5 marks)

(d) Weight decay (L2 penalty)

(5 marks)

Q&A